

App Development Project Report

IIT Madras BS Degree

Trekking Management Application (Mad I)

1. Student Details

Name: Ankush Maurya

Email: 23f3003103@ds.study.iitm.ac.in

Roll Number: DS23F3003103

About Me:

I am a Diploma-level student at Indian Institute of Technology Madras, pursuing Bachelor of Science in Data Science and Applications. I am interested in data science, AI, ML, programming and developing practical solutions through technology.

2. Project Details

Project Title: Placement Portal Application

Problem Statement

The Trekking Management Application is developed to streamline the management of trekking events by providing a centralized platform for **Administrators, Trekking Staff, and Participants**. Traditional trekking management often relies on manual registrations, paper records, and inefficient communication, making it difficult to organize events and track participant information. This application solves these challenges by enabling organizers to create trekking events, participants to register and manage bookings, and administrators to oversee the entire system efficiently. The platform improves coordination, reduces manual effort, and ensures a seamless trekking experience.

Approach

The application is built using **Flask** as the backend framework with a modular architecture to ensure maintainability and scalability. **SQLite** is used as the database, while **SQLAlchemy** handles database operations through ORM. The frontend is developed using **HTML, CSS, Bootstrap, and Jinja2 templates** to provide a responsive and user-friendly interface.

The system supports **role-based access** for three types of users:

- **Administrator:** Manages users, trekking Staff, trekking events, registrations, and monitors the overall system.
- **Trek Staff:** Creates trekking events, specifies routes, schedules, participant limits, and manages registered participants.
- **Participant/User:** Creates a profile, browses available treks, registers for treks, views booking history, and receives trek updates.

The application integrates features such as authentication, event management, participant registration, booking history. This approach ensures efficient management of trekking activities while providing a smooth and organized experience for both organizers and participants.

3. AI/LLM Declaration

I Ankush, declare that this project was developed primarily by my own learning, coding and implementation efforts.

AI was used only for the following purposes.

- Debugging code and understanding errors.
- Explanations of Flask, jinja and SQLAlchemy concepts and understanding some advanced backend routes.
- Getting help for styling so that the UI looks nice.
- Documentation formatting and organizing codes.

Based on my use, I estimate that AI/LLM contributed about 15-20% of the total work, mainly for debugging support, documentation formatting, and explanations of errors. I followed the academic guidelines of the Indian Institute of Technology regarding responsible use of AI. I confirm that all Implementation logic was done by me and AI used only for supporting tools.

4. Technologies and Frameworks Used

Technology/Library	Purpose
Flask	Core backend web framework.
SQLAlchemy	Object relational Mapper for SQLite Database
SQLite	Lightweight local database for storing user data, Placement drives, Applications, etc.
Jinja2	Template engine for rendering dynamic HTML pages
HTML	Structure the content of all web pages
Bootstrap	Frontend styling and responsive UI design
CSS	Some extra styling for all web pages to look good

5. Database Schema / ER Diagram

Tables:

1. **User** – Store user login credentials and their roles
(Id, fullname, email, password, role, status, joined_date)
2. **Staff_Profile** – Store students profile details
(Id, user_id, phone, about, location, experience)
3. **Trekker_Profile** – Store companies profile details
(Id, user_id, phone, about, location, trekking_experience)
4. **Treks** – Store placement drives details
(Id, trekname, location, difficulty, duration, slots, start_date, end_date, sname, staff_id, description, location, status)
5. **Bookings** – Store application details and status
(Id, user_id, trek_id, booking_date, status)

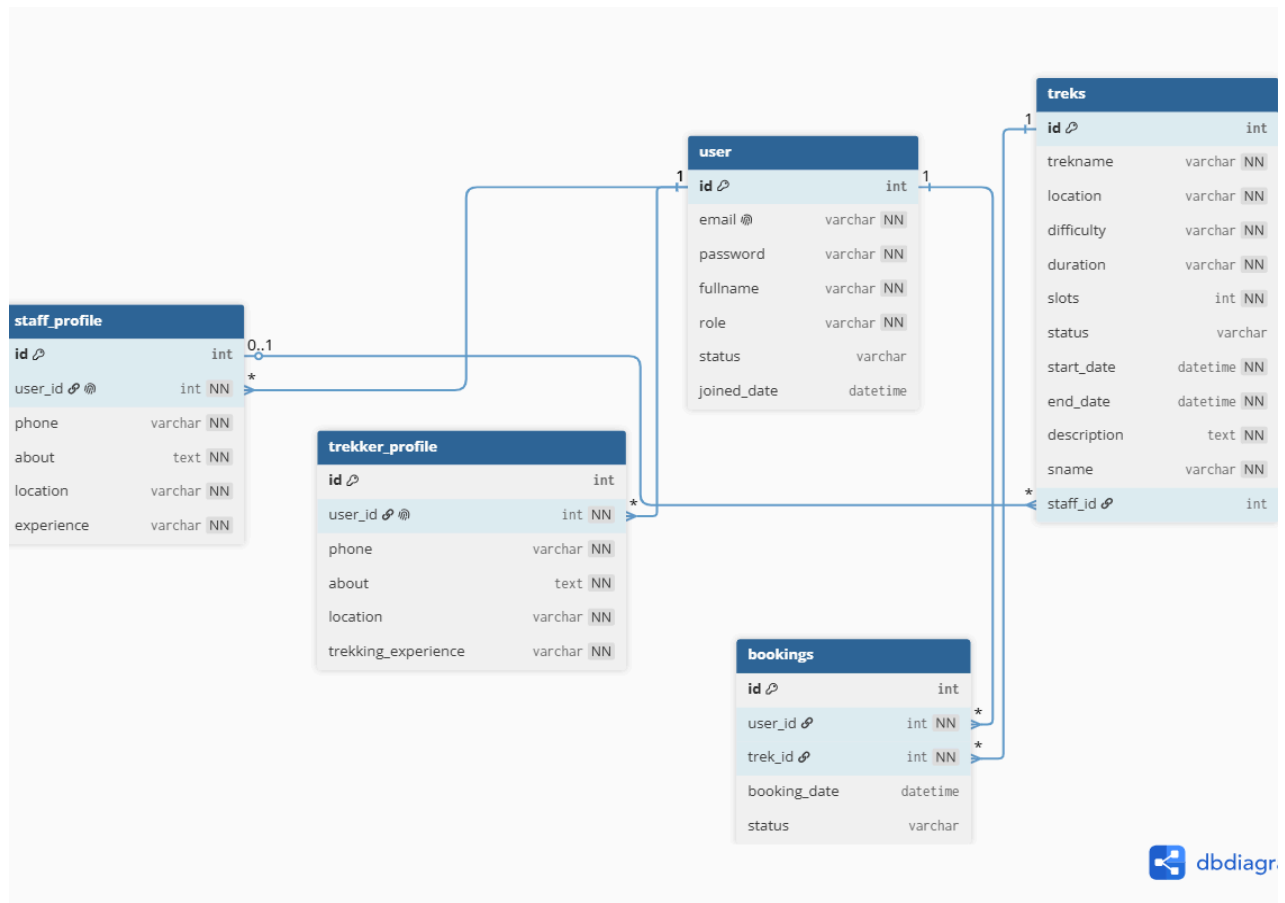
Relationships:

1. One-to-One:

- User → Staff
- User → Trekker

2. One-to-Many:

- User → Booking
- Trek → Booking
- Staff → Trek



(db made by dbdiagram.io)

6. API Resource Endpoints

Authentication and User Management

Routes	Methods	Description
/register	POST	Register new user (staff/Trekker)
/login	POST	Login for admin, staff, trekker

Admin Dashboard and Control

Routes	Methods	Description
/admin	GET	Admin Dashboard
/approve_booking/<id>	GET	Admin can approve booking
/reject_booking/<id>	GET	Admin can reject booking
/admin_trek	GET	Admin can manage trek
/admin_add_trek	POST	Admin can add new trek
/admin_manage_staff	GET	Admin can manage staff
/approve_staff/<staff_id>	GET	Admin can approve staff
/blacklist_staff/<staff_id>	GET	Admin can blacklist staff
/restore_staff/<staff_id>	GET	Admin can restore staff again
/admin_manage_trekker	GET	Admin can manage trekker
/admin_blacklist_trekker/<id>	GET	Admin can blacklist trekker
/admin_manage_booking	GET	Admin can manage booking

/admin_blacklist_booking/<id>	GET	Admin can blacklist booking
-------------------------------	-----	-----------------------------

Staff Dashboard and Control

Routes	Methods	Description
/staff/<id>	GET	Staff Dashboard
/staff/<staff_id>/manage_trek/<trek_id>	GET/POST	Staff can manage Trek
/staff_profile/<staff_id>	GET	Company profile details
/staff_edit_profile/<staff_id>	POST	Staff can edit profile

Trekker Dashboard and Control

Routes	Methods	Description
/trekker/<trekker_id>	GET	Trekker Dashboard
/trekker/<trekker_id>/view_trek/<trek_id>	GET	Trekker can view particular trek
/trekker/<trekker_id>/book_trek/<trek_id>	GET	Trekker can book on a trek
/trekker/<trekker_id>/bookings	GET	Trekker can view all their bookings
/trekker/<trekker_id>/profile	GET	Trekker's Profile details
/trekker/<trekker_id>/profile/edit	POST	Trekker can edit profile
/trekker/<trekker_id>/history	GET	Trekker's History

7. Architecture and Features (Optional)

Architecture Overview:

- **app.py** — Main Flask application entry points
- **/models** — Database models using SQLAlchemy
- **/auth** — Modular structure containing routes, utilities, and backend logic
- **/database** — Database configuration and initialization
- **/templates** — Jinja2 HTML template for all pages
- **/statics** — CSS file

Implemented Features:

User registration and login

Three roles: **Admin, Staff, Trekker/User**

Admin pre-exist (No admin registration)

Admin Features:

Admin can:

- View Total number of Treks, total number of Staff, total number of users/trekkers, and total number of bookings
- Create and manage treks routes
- Approve or Reject staff registration
- Assign staff to treks
- Deactivate or blacklist trekkers/staff
- View all bookings

Staff Features:

Staff can:

- View assigned treks by admin
- View number of registered trekker per trek
- Update trek slots and status
- View and manage participant list
- Mark trek as started/completed
- Edit profile details

Trekker Features:

Trekker can:

- Edit profile details
 - View all available treks, booked treks and trek status
 - Book treks and search treks
 - View booking status
 - View trekking history
-

8. Video Presentation

Drive Link: [Click Here](#)

(Accessible to all with "View" permission.)